

Mixturas Gaussianas

Lucía Sánchez Caparrós
Lucía Muñoz del Río
Zhibo Daniel Chen
María Victoria Maldonado Bao

0611565598@uma.es
Ciberseguridad e Inteligencia Artificial. Universidad de Málaga.

1 Introducción

Este informe explica el funcionamiento del algoritmo de mixturas gaussianas para un dataset real de **10GB** con millones de **partidas de ajedrez** registradas. Se comentará el método de selección elegido para hallar el número de componentes (k) óptimo, el preprocesamiento de los datos, así como un **análisis** de los resultados obtenidos para varios subconjuntos teniendo en cuenta las columnas originales, la versión extendida y calculando **PCA**.

2 Descripción de los Datos

2.1 Conjunto de Datos Original

El dataset procesado guarda información sobre partidas de ajedrez del mes de enero de 2025. En total, el archivo ocupa aproximadamente **10 GB** en formato **CSV**. Sus **atributos** son los siguientes:

- **ID** (cadena): identificador único de la partida.
- **White** (cadena): nombre de usuario del jugador con piezas blancas.
- **Black** (cadena): nombre de usuario del jugador con piezas negras.
- **Result** (cadena): resultado de la partida (“1-0”, “0-1” o “1/2-1/2”).
- **WhiteElo** (entero / “?”): puntuación Elo del jugador blanco. El valor “?” indica que no está registrado y se trata como 0.
- **BlackElo** (entero / “?”): puntuación Elo del jugador negro.
- **TimeControl** (cadena): ritmo de juego en formato x+y (segundos totales + incremento por jugada).
- **ECO** (cadena): código estandarizado de la apertura jugada.
- **Opening** (cadena): nombre de la apertura.
- **Termination** (cadena): motivo de finalización de la partida.
- **MovesCount** (entero): número total de jugadas de la partida.

Para el clustering se utilizan únicamente las columnas *WhiteElo*, *BlackElo*, *MovesCount* y *TimeControl* (esta última solo como selector de subconjunto, sin modificarla).

2.2 Subconjuntos Generados

A partir del dataset original se generan cuatro subconjuntos de trabajo:

- **Subconjunto 60+0 (original)**: partidas con ritmo de 60 segundos sin incremento, usando *WhiteElo*, *BlackElo* y *MovesCount* estandarizados.
- **Subconjunto 600+0 (original)**: partidas con ritmo de 600 segundos sin incremento, con los mismos atributos estandarizados.
- **Subconjunto 60+0 (extendido)**: igual que el anterior añadiendo *GameElo* y *EloDiff* como atributos calculados, que explicaremos posteriormente.
- **Subconjunto 600+0 (extendido)**: análogo al anterior para ritmo de 600 segundos.

3 Metodologías y Estrategias Utilizadas

3.1 Preprocesamiento por Chunks

Dado un dataset tan grande, no es viable cargarlo cada vez en RAM; por eso se realiza un procesamiento por bloques (**chunks**), en este caso de 50.000 filas. Se utiliza *partial_fit()* y *StandardScaler* para estandarizar los datos, además se limpian valores; todo dentro de la función *procesar_chunk()*. Lo más destacable es que los subconjuntos no se guardan en variables, sino en archivos locales **CSV**, ya que tenerlos cargados siempre en RAM sería muy pesado y no todos los equipos cuentan con extensa memoria. De esta manera, siempre que se necesite acceder al subconjunto, se leerá del archivo local y se cargará momentáneamente en RAM.

3.2 Método de selección BIC (Bayesian Information Criterion)

El método BIC nos permite determinar el **número óptimo de componentes**, buscando el mayor equilibrio entre la **bondad del ajuste** y la **simplicidad** para evitar el **overfitting**.

El BIC se calcula mediante la siguiente fórmula:

$$BIC = k \ln(n) - 2 \ln(\hat{L})$$

Donde:

$\hat{L}$: Es el valor maximizado de la función de verosimilitud del modelo.

n : Es el número de observaciones (tamaño de la muestra).

k : Es el número de componentes estimados por el modelo.

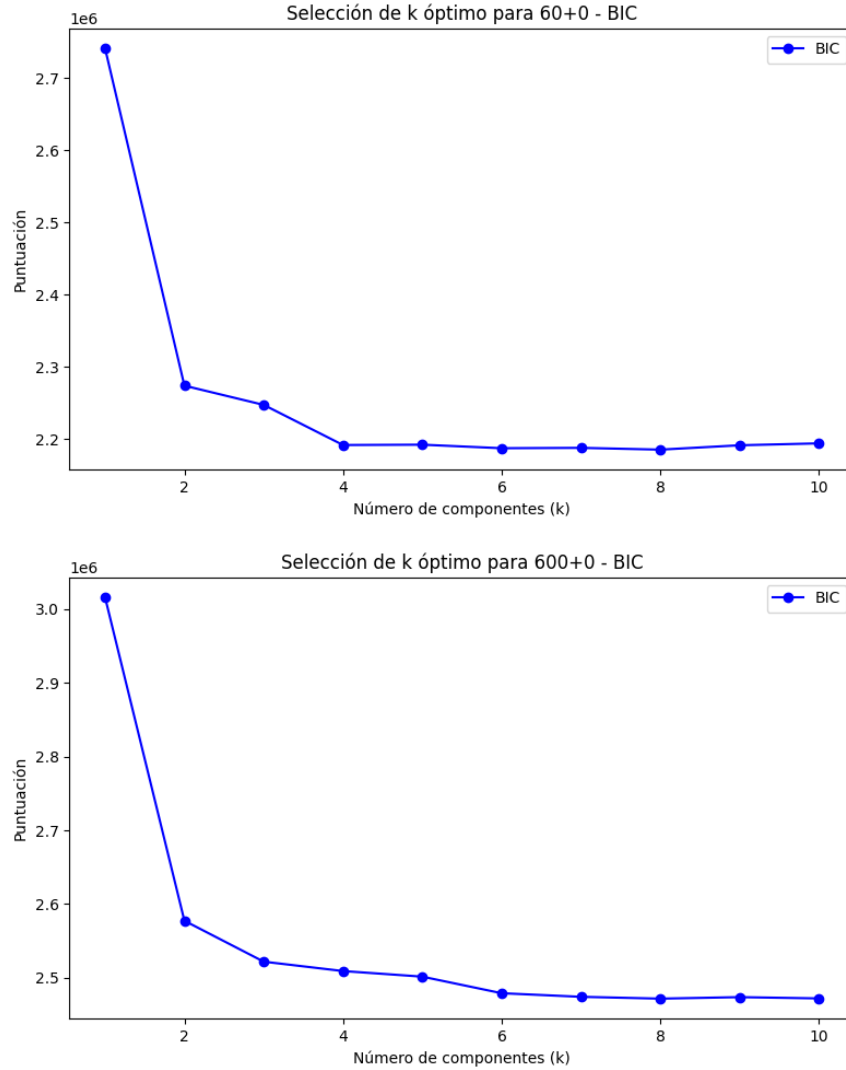


Fig. 1. Imágenes generadas por el script Python

3.3 Algoritmo de mixturas gaussianas

Este es un algoritmo de **clustering**, es decir, trata de clasificar en grupos los datos en el espacio; pero con la diferencia de que utiliza **probabilidades**: a un punto le asigna una probabilidad de pertenecer a un cluster o a otro. Además, hace uso de las campanas de Gauss para envolver los datos, las va ajustando y modifica su forma circular a **elipses**.

1. **Inicialización**: estimar o inventar los valores iniciales de los parámetros de las k campanas de Gauss: **medias, covarianzas y pesos** (μ_k, Σ_k, π_k)
2. **El Paso E (Expectation - Esperanza)**: usando las posiciones actuales de las campanas, calcular la responsabilidad (r_{ik}) dato a dato del dataset
3. **El Paso M (Maximization - Maximización)**: actualizar los parámetros de las campanas para que se adapten a la realidad de los puntos ponderados ($\mu_k^{new}, \Sigma_k^{new}, \pi_k^{new}$).

4. **Comprobación de la Convergencia:** calcular el valor de la **Log-Verosimilitud** total del modelo y comprobar si la puntuación apenas ha cambiado desde la iteración anterior. En caso contrario, volver al paso 1.

NOTA: en el código se utilizan muestras representativas de 500.000 filas (10.000 para las gráficas) elegidas aleatoriamente mediante una semilla para no extender excesivamente los tiempos de ejecución y hacer las gráficas interpretables.

3.4 Transformación de Características

La **transformación de características** consiste en "mezclar" las características o atributos de un conjunto de datos para conseguir así **reducir su dimensionalidad** y hacer los cálculos posteriores mucho más **ligeros computacionalmente**.

Para ello, se realiza un **análisis de componentes principales (PCA)**; es decir, se calculan cada una de las componentes del espacio de coordenadas con un porcentaje concreto de **explicabilidad de la varianza total** de los atributos originales.

En este caso, se ha realizado un PCA de **0.95** y **0.80** para cada uno de los subconjuntos extendidos.

4 Implementación

El código está programado en *Python*, las librerías utilizadas son:

- **pandas** y **numpy**.
- **scikit-learn**: *StandardScaler*, *GaussianMixture*, *PCA*.
- **matplotlib**: generación de gráficas.
- **gc**: liberación manual de memoria entre bloques.

```
CHUNK_SIZE = 50_000
CSV_PATH = "lichess_games.csv"
COLS = ['WhiteElo', 'BlackElo', 'MovesCount', 'TimeControl']
NUM_COLS = ['WhiteElo', 'BlackElo', 'MovesCount']

def procesar_chunk(chunk):
    chunk['WhiteElo'] = chunk['WhiteElo'].replace('?', 0).astype(int)
    chunk['BlackElo'] = chunk['BlackElo'].replace('?', 0).astype(int)
    chunk.dropna(subset=COLS, inplace=True)
    return chunk

scaler = StandardScaler()
for chunk in pd.read_csv(CSV_PATH, usecols=COLS, chunksize=CHUNK_SIZE):
    chunk = procesar_chunk(chunk)
    scaler.partial_fit(chunk[NUM_COLS])

OUTPUT_600 = "lichess_600.csv"
OUTPUT_60 = "lichess_60.csv"

for f in [OUTPUT_600, OUTPUT_60]:
    if os.path.exists(f): os.remove(f)

for i, chunk in enumerate(pd.read_csv(CSV_PATH, usecols=COLS, chunksize=CHUNK_SIZE)):
    chunk = procesar_chunk(chunk)
```

```

scaled_num = scaler.transform(chunk[NUM_COLS])
scaled = pd.DataFrame(scaled_num, columns=NUM_COLS, index=chunk.index)
scaled['TimeControl'] = chunk['TimeControl'].values

mask_600 = chunk['TimeControl'] == '600+0'
mask_60 = chunk['TimeControl'] == '60+0'

if mask_600.any():
    scaled.loc[mask_600, NUM_COLS].to_csv(OUTPUT_600, mode='a', index=False,
        header=not os.path.exists(OUTPUT_600))
if mask_60.any():
    scaled.loc[mask_60, NUM_COLS].to_csv(OUTPUT_60, mode='a', index=False,
        header=not os.path.exists(OUTPUT_60))

if i % 20 == 0:
    gc.collect()

```

Listing 1.1. Preprocesamiento de los datos de "lichess_games"

El preprocesamiento aplica la lógica que se comentó en la sección de metodologías y estrategias utilizadas: se recorre el dataset por chunks de 50.000 filas, **estandarizando** los datos usando *StandardScaler* y limpiando valores, como cambiar los signos de interrogación '?' por 0. **Se definen los archivos de salida** para los subconjuntos y se hace una primera **comprobación** para no duplicarlos, borrándolos en su caso. Luego, **comienzan las iteraciones**, extrayendo las filas pertenecientes a '600+0' y '60+0'. Finalmente, el programa guarda de forma local los **subconjuntos** definidos por los valores mencionados de la columna *TimeControl*.

En la segunda parte de la práctica, se pide añadir dos nuevas columnas *GameElo* y *Elodiff*, que son **calculadas** de otros atributos del dataset. De esta manera, respectivamente, se calcularán: $CEIL((WhiteElo + BlackElo)/2)$ y $|WhiteElo - BlackElo|$. Una vez agregadas las nuevas columnas, se repetirá el preprocesamiento de la misma manera.

```

gmm_final = GaussianMixture(n_components=k_opt, random_state=42)
gmm_final.fit(df_60_sample)
etiquetas_duras = gmm_final.predict(df_60_sample)

```

Listing 1.2. "Implementación del algoritmo de mixturas gaussianas"

Aplicar el algoritmo mediante la librería es muy fácil:

1. Se define el modelo haciendo uso del número óptimo de componentes (k) calculado previamente y de una semilla aleatoria.
2. Se entrena el modelo con la muestra del subconjunto correspondiente, en este caso '60+0'.
3. Se obtiene la clasificación de puntos a cada cluster con *.predict()*, en este caso se calculan etiquetas para **hard clustering**, pero se puede aplicar **soft clustering** con *.predict_proba()*.

5 Resultados Obtenidos

Se calculará el algoritmo de mixturas gaussianas para cada uno de los subconjuntos siguiendo secuencialmente las metodologías y estrategias explicadas en la sección anterior.

5.1 Basado en el Conjunto de Datos Inicial

Subconjunto 60+0 Aplicando el algoritmo de mixturas gaussianas para $k = 8$ (según el método BIC) sobre el subconjunto 60+0 con los atributos *WhiteElo*, *BlackElo* y *MovesCount* estandarizados, se obtiene el resultado de la Figura 2.

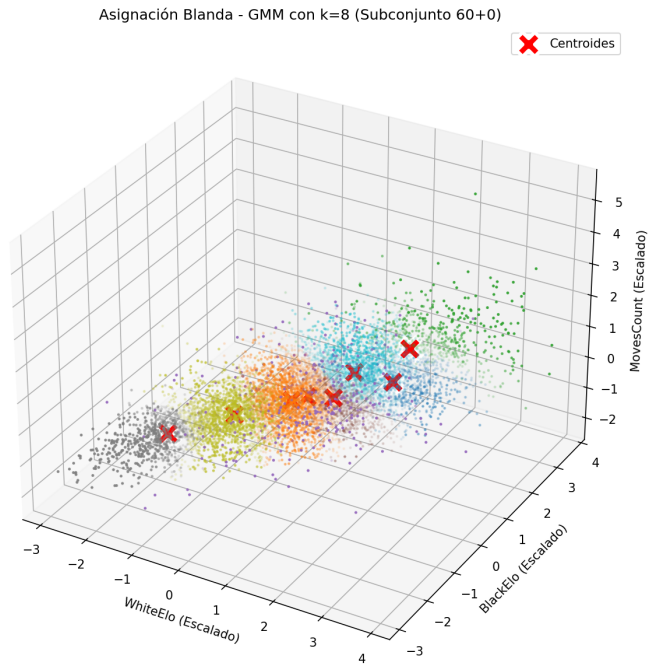
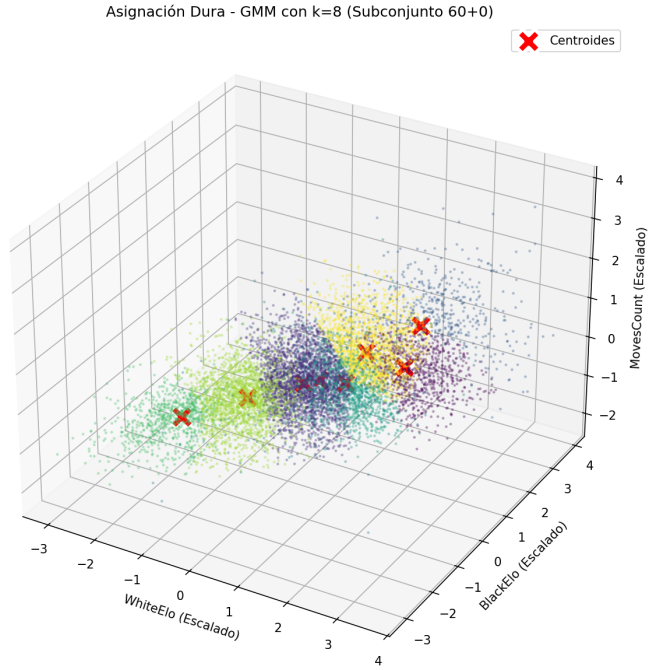


Fig. 2. Clústeres resultantes para el subconjunto 60+0

En la asignación dura (hard clustering), la nube de puntos muestra una estructura diagonal pronunciada en el plano WhiteElo–BlackElo, evidenciando la fuerte correlación entre ambos Elos: los jugadores fuertes tienden a enfrentarse entre sí y los débiles también. Los clústeres se distribuyen a lo largo de esta diagonal, segmentando principalmente por nivel de juego, con algunos grupos que también incorporan variación en

el eje MovesCount. Los centroides se concentran en la zona central-alta de la diagonal, donde la densidad de partidas es mayor, correspondiente a jugadores de nivel intermedio-avanzado. Los clústeres del extremo inferior izquierdo agrupan partidas con Elos muy bajos (jugadores sin clasificar o principiantes), mientras que los del extremo superior derecho corresponden a partidas entre jugadores más experimentados.

En la asignación blanda (soft clustering), la diferencia más notable es la mayor mezcla de colores en la zona central de la nube, donde los Elos son intermedios. Esto refleja correctamente la naturaleza probabilística del GMM: los puntos en zonas de transición entre clústeres tienen probabilidades repartidas entre varios grupos en lugar de una asignación forzada. Los puntos con color más intenso corresponden a partidas que pertenecen con alta probabilidad a un único clúster, mientras que los más desvaídos representan casos ambiguos situados en las fronteras entre componentes gaussianas.

Subconjunto 600+0 Para el subconjunto 600+0 con los mismos atributos y $k = 8$, se obtiene el resultado de la Figura 3.

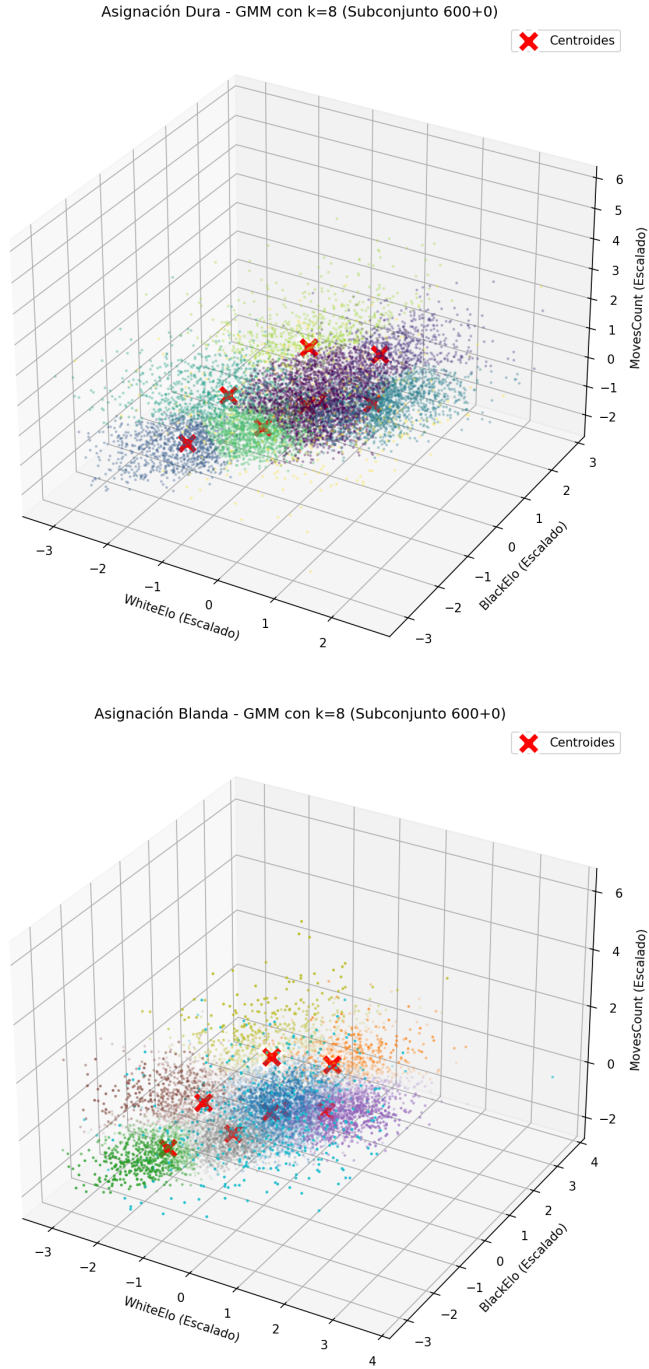


Fig. 3. Clústeres resultantes para el subconjunto 600+0

Los resultados son cualitativamente similares a los del subconjunto 60+0, confirmando que la estructura de agrupamiento es robusta e independiente del ritmo de juego. Sin embargo, se aprecian dos diferencias notables: la nube de puntos es más compacta en el plano WhiteElo–BlackElo, con menos dispersión hacia valores extremos de Elo, lo que sugiere que el ritmo de 600 segundos atrae a un perfil de jugador más

homogéneo en cuanto a nivel. Además, el eje MovesCount presenta una mayor dispersión vertical, coherente con el hecho de que las partidas más largas en tiempo permiten más jugadas antes de que el tiempo expire. Los centroides mantienen una distribución análoga a la del subconjunto anterior. En la asignación blanda, la zona de solapamiento entre clústeres es ligeramente menor que en 60+0, lo que indica que el modelo gaussiano ajusta mejor las componentes cuando los jugadores tienen perfiles más estables.

5.2 Basado en el Conjunto de Datos con Columnas Añadidas

Subconjunto 60+0 extendido Al añadir *GameElo* y *EloDiff*, el espacio pasa a **5 dimensiones**. El método BIC sigue devolviendo $k = 8$. La separación entre clústeres es mucho menos clara, dada la dependencia lineal entre atributos. Se obtiene el resultado de la Figura 4.

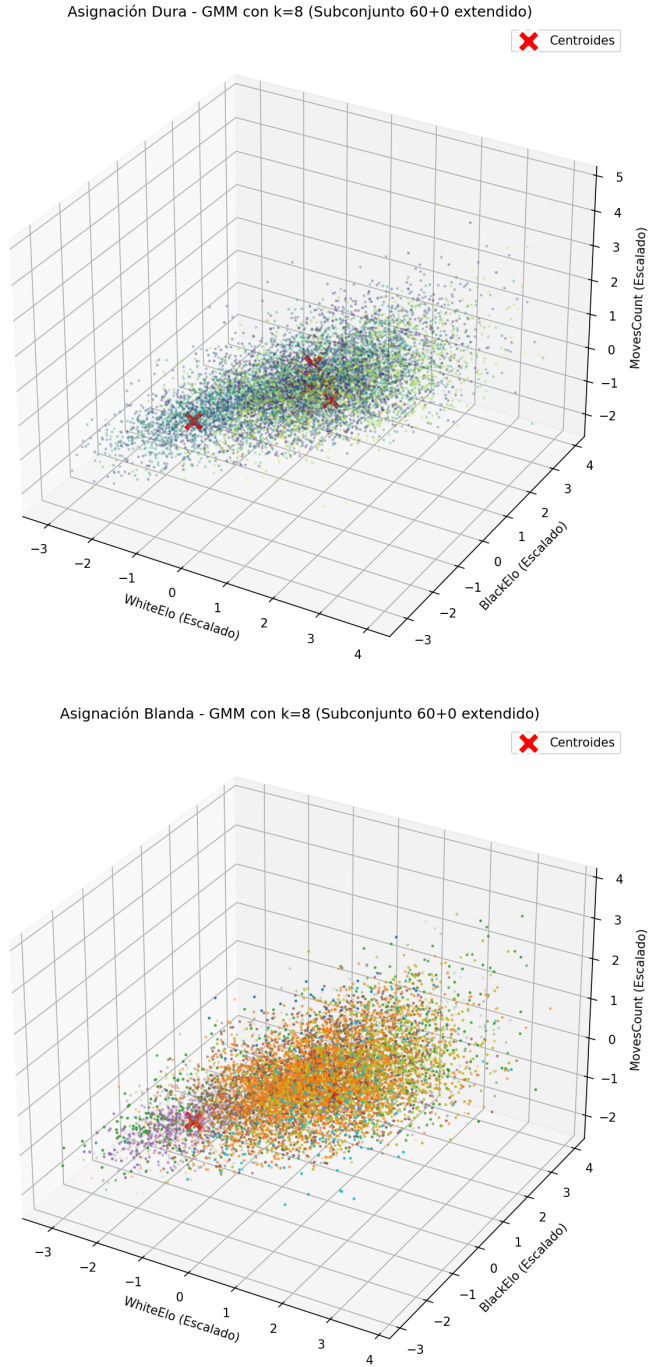


Fig. 4. Clústeres resultantes para el subconjunto 60+0 extendido

La adición de GameElo y EloDiff introduce una colinealidad severa en el espacio de características, ya que ambas columnas se calculan directamente a partir de WhiteElo y BlackElo mediante operaciones lineales simples. El efecto en el clustering es inmediato: los clústeres pierden casi toda su separación visual, apareciendo como una masa compacta y altamente solapada donde resulta muy difícil distinguir fronteras

entre grupos, como muestran claramente ambas gráficas de la Figura 4. Los centroides se concentran en una región muy reducida del espacio, con apenas separación entre ellos, lo que indica que el GMM tiene dificultades para encontrar elipsoides de covarianza bien diferenciados en un espacio donde varias dimensiones aportan información redundante. En la asignación blanda, prácticamente todos los puntos presentan colores mezclados, confirmando que el modelo asigna probabilidades repartidas entre múltiples componentes sin una componente dominante clara.

PCA 0.95.

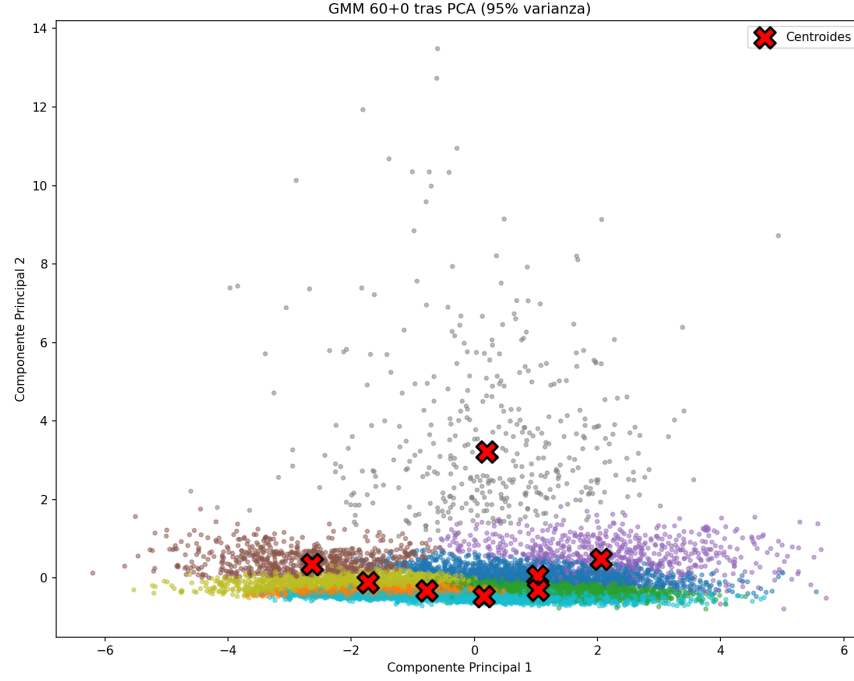


Fig. 5. Clústeres resultantes para el subconjunto 60+0 extendido con PCA 0.95

Tras aplicar PCA reteniendo el 95% de la varianza, los datos se proyectan sobre dos componentes principales. La Componente Principal 1 concentra la mayor parte de la variabilidad y está dominada por la magnitud absoluta de los Elos (WhiteElo, BlackElo y GameElo son fuertemente colineales), generando una densa franja horizontal que representa el gradiente de nivel de juego de izquierda (jugadores débiles) a derecha (jugadores fuertes). La Componente Principal 2 recoge la variabilidad residual, principalmente asociada a MovesCount y EloDiff, lo que explica la dispersión vertical observada. Los clústeres se distribuyen a lo largo del eje horizontal con cierta superposición, visible en la mezcla de colores, especialmente en la zona central. Los puntos que se alejan notablemente hacia valores altos de la Componente 2 son outliers correspondientes a partidas con un número de jugadas excepcionalmente elevado. Los centroides se alinean sobre el eje dominante, reflejando que la separación entre grupos ocurre principalmente en función del nivel Elo y no de la duración.

PCA 0.80.

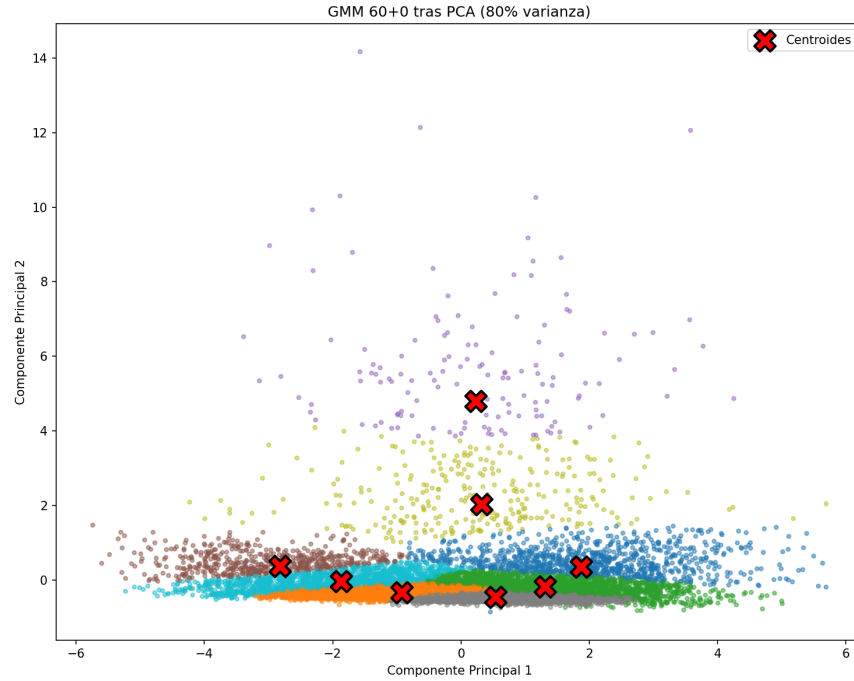


Fig. 6. Clústeres resultantes para el subconjunto 60+0 extendido con PCA 0.80

Con el 80% de varianza retenida, el PCA reduce aún más la información preservada, lo que se traduce en una nube de puntos más compacta verticalmente respecto a la Figura 5. La estructura general se mantiene: franja horizontal dominada por el nivel Elo, con dispersión vertical residual. Los clústeres presentan un solapamiento ligeramente mayor que con PCA 0.95, especialmente en los extremos del eje horizontal, donde la pérdida del 15% adicional de varianza elimina matices que ayudaban a separar grupos de Elo muy alto o muy bajo. Los centroides siguen alineados sobre el eje principal. En general, la diferencia entre PCA 0.95 y PCA 0.80 es moderada, lo que indica que la quinta dimensión aportaba poca información adicional para el clustering, algo esperable dada la alta colinealidad entre atributos.

Subconjunto 600+0 extendido Los resultados son cualitativamente similares al caso 60+0 para el PCA. A continuación se muestran las gráficas con los resultados:

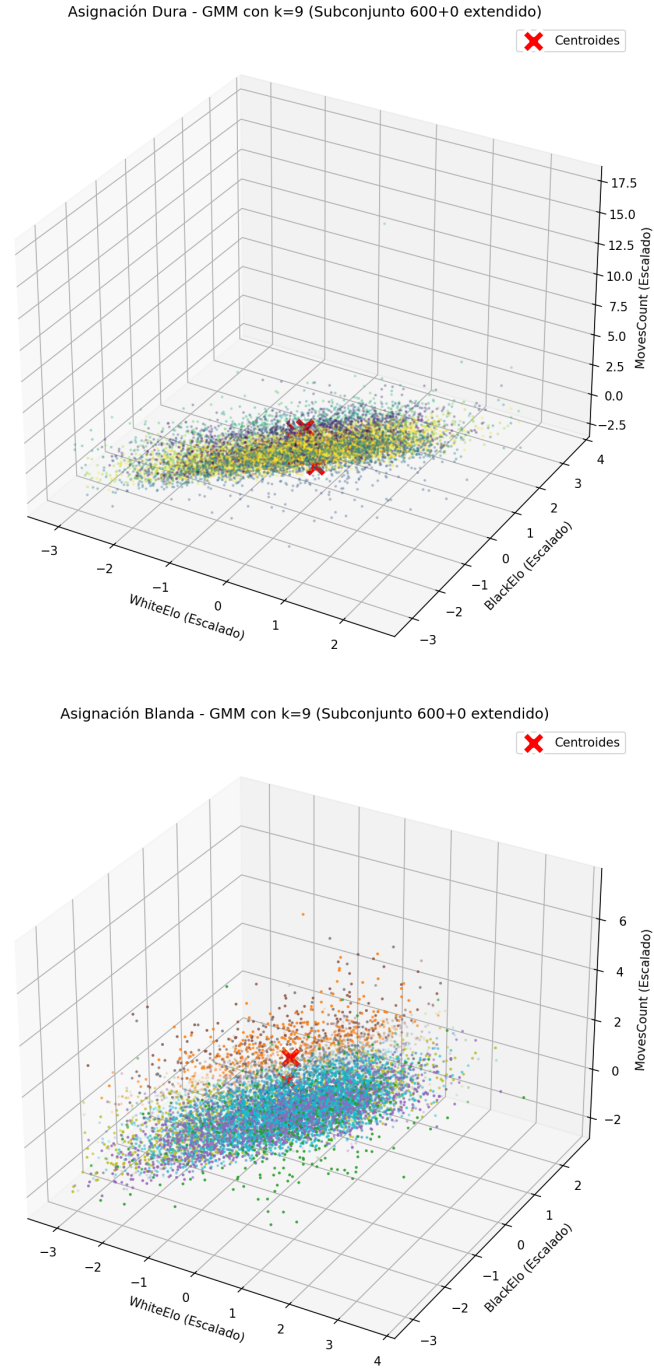


Fig. 7. Clústeres resultantes para el subconjunto 600+0 extendido

Los resultados son análogos a los del subconjunto 60+0 extendido, con la misma pérdida de separación entre clústeres causada por la colinealidad de los atributos derivados. La diferencia más llamativa aparece en el eje MovesCount de la asignación dura: se observan valores extremadamente altos (hasta 17.5 en escala estandarizada), que corresponden a partidas excepcionalmente largas en movimientos. Estos outliers distor-

sionan la escala del eje vertical y hacen que el grueso de los puntos aparezca comprimido en la parte baja del gráfico, dificultando aún más la interpretación visual. En la asignación blanda el solapamiento es total, confirmando que con 5 dimensiones colineales el GMM no logra establecer fronteras probabilísticas claras sin la ayuda del PCA.

PCA 0.95 y 0.80.

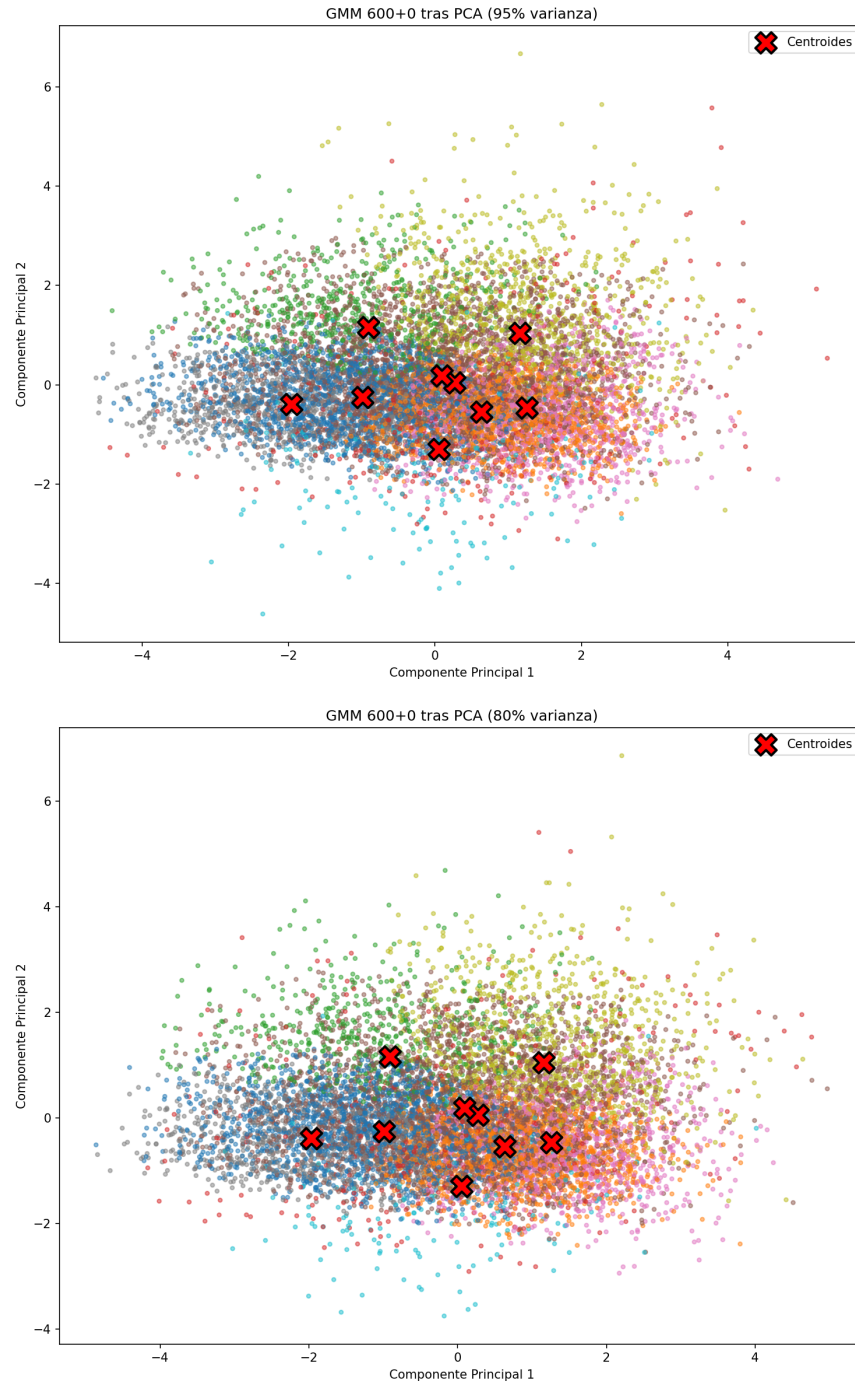


Fig. 8. Clústeres resultantes para el subconjunto 600+0 extendido con PCA

A diferencia de los resultados del subconjunto 60+0 extendido con PCA, en este caso las gráficas muestran una separación entre clústeres notablemente mejor. Los puntos se distribuyen en una nube elíptica más amplia en ambas dimensiones, con los centroides claramente separados a lo largo de ambos ejes. Esto sugiere que para el ritmo de 600 segundos, las componentes principales capturan una estructura de variabilidad más

rica, probablemente porque en partidas más largas las diferencias de nivel (EloDiff) y el número de jugadas (MovesCount) aportan información más discriminativa. La diferencia entre PCA 0.95 y PCA 0.80 es mínima en este caso: ambas gráficas son prácticamente idénticas, lo que indica que la varianza está muy concentrada en las dos primeras componentes y reducir del 95% al 80% apenas elimina información relevante para el clustering.

6 Discusión

6.1 Comparación Entre Subconjuntos

Al observar los resultados obtenidos para los subconjuntos originales de 60+0 (ritmo bala) y 600+0 (ritmo rápido), se nota que el comportamiento general del agrupamiento es bastante similar. En ambos casos, el método BIC determina que el número óptimo de componentes es $k = 8$. Las gráficas tridimensionales (Figuras 2 y 3) muestran que los clústeres se distribuyen de manera análoga a lo largo de los ejes de las puntuaciones Elo y la cantidad de movimientos. Esto sugiere que, independientemente del control de tiempo, los patrones de emparejamiento (jugadores de Elo similar) y la distribución de la duración de las partidas mantienen una estructura subyacente común que el algoritmo de mixturas gaussianas (GMM) logra identificar de forma consistente.

6.2 Impacto de los Atributos Calculados

La adición de las variables GameElo y EloDiff incrementa la dimensionalidad del espacio de características a cinco dimensiones. Tal y como evidencian los resultados, esto provoca que la separación entre los clústeres sea mucho menos clara debido a una fuerte dependencia lineal entre los atributos. Esta colinealidad ocurre porque los nuevos atributos se calculan directamente mediante sumas y restas simples de WhiteElo y BlackElo. En modelos basados en covarianza como GMM, esta redundancia deforma el espacio, traduciéndose visualmente en agrupaciones altamente superpuestas (Figuras 4 y 7). Además, la complejidad geométrica introducida por esta redundancia provoca que, para el subconjunto de 600+0 extendido, el modelo asigne un número óptimo de componentes ligeramente distinto ($k = 9$) en un intento de adaptarse a la nueva densidad de los datos.

6.3 Impacto del PCA

Para mitigar la redundancia y la colinealidad generada en la versión extendida, la aplicación del Análisis de Componentes Principales (PCA) resulta fundamental. Al retener el 95% y el 80% de la varianza total de los atributos originales, se observa en las Figuras 5, 6 y 8 que la mayor parte de la información se comprime eficientemente en solo dos dimensiones principales. La Componente Principal 1 absorbe la inmensa mayoría de la variabilidad de los datos (probablemente capturando la magnitud absoluta de las puntuaciones Elo), generando una densa franja horizontal. Reducir la explicabilidad al 80% compacta aún más esta nube de puntos, limitando la dispersión en el eje vertical. El PCA demuestra ser una herramienta eficaz para reducir la carga computacional y eliminar correlaciones, aunque como contrapartida genera que los centroides se alineen sobre un eje dominante, perdiendo parte de la interpretabilidad intuitiva que ofrecían las variables originales.

6.4 Limitaciones

El análisis realizado presenta varias limitaciones que deben tenerse en cuenta: Restricciones computacionales y de visualización: Debido al masivo tamaño del conjunto de datos original (10 GB), la generación de gráficas requirió el uso de muestras representativas reducidas (10.000 filas). Aunque esto es estadísticamente válido y necesario para evitar tiempos de ejecución excesivos, la visualización tridimensional en proyecciones estáticas puede hacer que los clústeres parezcan más mezclados de lo que están matemáticamente en el hiperespacio. Naturaleza de las distribuciones: GMM asume que la topología de los datos puede modelarse

mediante campanas de Gauss (distribuciones normales). Sin embargo, variables como MovesCount suelen presentar asimetrías (colas largas hacia la derecha, ya que las partidas pueden alargarse indefinidamente pero no pueden tener menos de 0 movimientos). Es probable que el modelo necesite emplear varios componentes (aumentando la k) simplemente para ajustarse a estas colas asimétricas, y no porque representen perfiles de jugadores verdaderamente distintos. Problema de colinealidad no resuelto antes del GMM: Haber ejecutado el clustering espacial directamente sobre variables linealmente dependientes (versión extendida) sin un PCA previo penaliza la capacidad del modelo para hallar elipsoides óptimos, explicando la alta fusión visual de los clústeres en las Figuras 4 y 7.

7 Conclusiones

En esta práctica se ha implementado el algoritmo de mixturas gaussianas sobre un **dataset real** de gran tamaño:

1. El procesamiento por chunks con *partial_fit()* y el almacenamiento local de los subconjuntos permite manejar **10 GB de datos** con *RAM* limitada, sin pérdida de precisión en la normalización.
2. El **método BIC** ha demostrado ser una herramienta fiable para seleccionar el número óptimo de componentes, identificando $k = 8$ de forma consistente para todos los subconjuntos originales. Para el subconjunto **600+0 extendido** selecciona $k = 9$, lo que refleja que la colinealidad introducida por los atributos derivados aumenta artificialmente la complejidad percibida del espacio, obligando al modelo a usar una componente adicional para adaptarse a la nueva geometría de los datos.
3. La adición de **GameElo** y **EloDiff**, a pesar de enriquecer semánticamente la descripción de cada partida, penaliza severamente la capacidad del GMM para encontrar agrupaciones bien diferenciadas. Al ser columnas calculadas directamente mediante sumas y restas de WhiteElo y BlackElo, introducen una colinealidad lineal que deforma las matrices de covarianza del modelo, haciendo que los clústeres se solapen casi completamente en el espacio de características extendido.
4. El **PCA** resulta imprescindible cuando se trabaja con el espacio extendido, ya que elimina la redundancia entre atributos y comprime la información relevante en dos componentes principales interpretables. Con PCA 0.95 se obtiene una separación de clústeres clara y visualmente significativa. La reducción adicional a PCA 0.80 apenas empeora los resultados en el subconjunto 600+0, donde la varianza está muy concentrada en las primeras componentes, pero sí introduce algo más de solapamiento en 60+0, lo que sugiere que este subconjunto requiere algo más de información para describir correctamente la estructura de sus grupos.

8 Uso de IA

Durante el desarrollo de la práctica se ha utilizado **Gemini** para los siguientes propósitos:

- **Código:** para generar las gráficas y realizar el PCA.
- **Redacción del informe:** revisión del informe en pos de comprobar que incluyera todos los puntos de la práctica y detectar errores gramaticales y de cohesión.



UNIVERSIDAD
DE MÁLAGA